This allows a single function template to accept both lvalue and rvalue parameters and is used by the `std::thread` constructor (sections 2.1 and 2.2) so that the supplied callable object can be moved into internal storage rather than copied if the parameter is an rvalue.

A.2. Deleted functions

Sometimes it doesn't make sense to allow a class to be copied. `std::mutex` is a prime example of this—what would it mean if you did copy a mutex? `std::unique_lock<>` is another—an instance is the one and only owner of the lock it holds. To truly copy it would mean that the copy also held the lock, which doesn't make sense. Moving ownership between instances, as described in section A.1.2, makes sense, but that's not copying. I'm sure you've met other examples.

The standard idiom for preventing copies of a class used to be to declare the copy constructor and copy assignment operator private and then not provide an implementation. This would cause a compile error if any code outside the class in question tried to copy an instance and a link-time error (due to lack of an implementation) if any of the class's member functions or friends tried to copy an instance:

```
class no_copies
{
public:
    no_copies(){}
private:
    no_copies(no_copies const&);
    no_copies& operator=(no_copies const&);
};
no_copies a;
no_copies b(a);
```

❶ No implementation

❷ Won't compile

With C++11, the committee realized that this was a common idiom but also realized that it's a bit of a hack. The committee therefore provided a more general mechanism that can be applied in other cases too: you can declare a function as *deleted* by adding `= delete` to the function declaration. `no_copies` can thus be written as